

R-Control IP4 – HTTP Legacy API (CGI)

Was ist das?

Eine sehr einfache HTTP-Schnittstelle: ein Aufruf im Browser oder per `curl` genügt, um zu schalten. Kein Anmelden, kein JSON, keine Kopfzeilen.

Diese Schnittstelle ist für die Rutenbeck Remote App und für bestehende Installationen gedacht. Für neue Projekte nehmen Sie besser die [HTTP/JSON-API](#) — sie ist abgesichert und kann deutlich mehr.

Wichtig: Diese Schnittstelle hat **keine Anmeldung**. Wer das Gerät im Netz erreicht, kann schalten. Aktivieren Sie sie nur in einem abgeschotteten Netz.

In 2 Minuten zum Ziel

Schritt 1: Einschalten

Weboberfläche → **Einstellungen** → **Schnittstellen** → **HTTP Legacy API** → Haken setzen, speichern.

Schritt 2: Schalten

Ausgang 1 umschalten — einfach im Browser aufrufen:

```
http://192.168.0.3/leds.cgi?led=1
```

Jeder Aufruf **schaltet um** (aus → ein → aus). Einen festen Zustand setzen Sie damit nicht; dafür nehmen Sie die HTTP/JSON-API.

Schritt 3: Zustand lesen

```
http://192.168.0.3/status.xml
```

Liefert ein kleines XML mit dem Zustand aller Ausgänge und der Temperatur.

Impuls — zum Beispiel ein Türöffner für 3 Sekunden

```
http://192.168.0.3/impl.cgi?ausg100:00:03normal
```

Gelesen wird das als: Ausgang **1**, Dauer **00:00:03**, Modus **normal** (Impuls EIN). Mit `reset` statt `normal` erhalten Sie einen Impuls AUS.

Beispiele

```
# Ausgang 2 umschalten
curl "http://192.168.0.3/leds.cgi?led=2"

# Zustand abfragen
curl "http://192.168.0.3/status.xml"

# Ausgang 1 für 5 Sekunden einschalten
curl "http://192.168.0.3/impl.cgi?ausg100:00:05normal"
```

Ab hier folgt der **vollständige Referenzteil** mit allen Details.

Diese Dokumentation beschreibt die **HTTP Legacy API** (CGI-Endpunkte) des R-Control IP4 aus Integrationsicht. Sie richtet sich an Kunden/Partner, die eine sehr schlanke, leicht skriptbare HTTP-Schnittstelle (z. B. für Home Assistant, SPS/SCADA, Testtools) benötigen.

Stand: Implementierung in `components/http_legacy_api` (Firmware-Repo). Die HTTP Legacy API ist bewusst **minimalistisch** und **nicht authentifiziert**.

1. Überblick

Zweck

- Einfaches Schalten eines Relais per HTTP GET (Toggle).
- Auslösen eines Zeitimpulses (Impuls EIN/AUS für eine definierte Dauer).
- Abruf eines kleinen XML-Statusdokuments (Relaiszustände + NTC-Status/Temperatur).

Eigenschaften

- Transport: HTTP (unverschlüsselt) oder HTTPS (sofern im Gerät aktiviert)
- Semantik: GET-Requests, Antworten als `text/plain` bzw. `application/xml`
- **Keine Authentifizierung / keine Autorisierung**

2. Aktivierung & Konfiguration

Die HTTP Legacy API ist standardmäßig deaktiviert und wird über die Gerätekonfiguration eingeschaltet.

2.1 Konfigurationsschlüssel

- `config.interfaces.http-legacy.enabled` (bool)
 - `true` = CGI-Endpunkte werden registriert
 - `false` = Endpunkte sind nicht verfügbar

2.2 Port / Basis-URL

Die Legacy-Endpunkte werden am selben HTTP-Server bereitgestellt wie UI/REST:

- HTTP-Port: `config.network.lan.eth0.http.port` (Default meist 80)
- HTTPS: `config.network.lan.eth0.https.enabled` (optional)

Wichtig:

- `config.interfaces.http-legacy.port` wird zwar in mDNS-TXT Feldern geführt, die Implementierung der CGI-Endpunkte selbst hängt jedoch am bestehenden HTTP-Server-Handle.
- Praktisch bedeutet das: Die Legacy-Endpunkte sind über die Basis-URL des Geräts erreichbar:
 - `http://<gerät>/leds.cgi`
 - `http://<gerät>/impl.cgi`
 - `http://<gerät>/status.xml`

2.3 mDNS/Service Discovery

Wenn mDNS aktiv ist, kann der `_http._tcp`-Service TXT-Records enthalten (u. a. `http_legacy=true/false`, `http_legacy_port=<port>`).

3. Relaiskanäle

Die Legacy-HTTP-API arbeitet mit einem **1-basierten** Kanalindex:

- `led=1` entspricht Relais 1
- ...

Die Firmware ermittelt intern die Anzahl der konfigurierten Ausgänge über `config.peripherals.switch_outputs[]`.

Für `status.xml` gilt:

- Es werden **immer 4 Kanäle** ausgegeben (`<led1>...</led1>` bis `<led4>...</led4>`). Wenn das Gerät weniger als 4 Ausgänge konfiguriert hat, werden fehlende Kanäle als `0` gemeldet.

4. Endpunkte

4.1 Toggle Relais

Request

- `GET /leds.cgi?led=<n>`

Parameter

- `led` (1..N)

Antwort

- Body: `OK\n` oder `ERR\n`
- Content-Type: `text/plain; charset=utf-8`
- Cache: `Cache-Control: no-store, no-cache, must-revalidate`

HTTP Statuscodes

- `200 OK` bei Erfolg
- `400 Bad Request` bei ungültigen Parametern
- `500 Internal Server Error` bei internen Fehlern (z. B. Relay-Schaltfehler)

Beispiel

```
curl "http://r-control-ip4.local/leds.cgi?led=2"
```

4.2 Impuls (Impuls EIN/AUS mit Dauer)

Request

- `GET /impl.cgi?ausg<kanal><HH:MM:SS><normal|reset>`

Bedeutung

- `normal` = Impuls EIN
- `reset` = Impuls AUS

Parameterformat

- Prefix: `ausg`
- `<kanal>`: 1..N (dezimal, ohne Trennzeichen)
- `<HH:MM:SS>`: exakt 8 Zeichen, z. B. `00:00:10`
 - `MM` und `SS` müssen im Bereich `00..59` sein
 - Gesamtzeit muss `> 0` sein
- Suffix: `normal` oder `reset`

Antwort

- Body: `OK\n` oder `ERR\n`
- Content-Type: `text/plain; charset=utf-8`

HTTP Statuscodes

- `200 OK` bei Erfolg
- `400 Bad Request` bei ungültigem Query-Format / ungültigem Kanal / ungültiger Dauer
- `500 Internal Server Error` bei internen Fehlern (z. B. Relay-Controller Fehler)

Beispiele

```
# Relais 1 für 10 Sekunden einschalten (Impuls ON)
curl "http://r-control-ip4.local/impl.cgi?ausg100:00:10normal"

# Relais 1 für 10 Sekunden ausschalten (Impuls OFF)
curl "http://r-control-ip4.local/impl.cgi?ausg100:00:10reset"
```

Hinweis: Der Query-String wird als kompletter Payload interpretiert (kein klassisches `key=value`).

4.3 Status (XML)

Request

- GET /status.xml

Antwort

- Content-Type: application/xml; charset=utf-8
- Cache: Cache-Control: no-store, no-cache, must-revalidate
- Body (Beispiel):

```
<response>
<led1>0</led1>
<led2>1</led2>
<led3>0</led3>
<led4>0</led4>
<pot0>22.50</pot0>
</response>
```

Feldbeschreibung

- <led1> .. <led4>: 0 oder 1
- <pot0>: Temperatur-/Statusfeld (Text oder Zahl)
 - Bei gültiger Temperatur: Temperatur in °C mit 2 Nachkommastellen (z. B. 22.50)
 - Bei NTC Out-of-Range LOW: unterbrochen
 - Bei NTC Out-of-Range HIGH: kurzschluss
 - Wenn keine Daten verfügbar: nicht angeschlossen

5. Fehlerverhalten

Die Legacy-HTTP-API signalisiert Fehler bewusst einfach:

- Response-Body: ERR\n
- Statuscode: typischerweise 400 (Client-Fehler) oder 500 (Server-Fehler)

5.1 Häufige Ursachen für 400 Bad Request

- Fehlender Query oder zu langer Query (internes Limit: 64 Zeichen)
- /leds.cgi:
 - led fehlt
 - led nicht numerisch oder außerhalb des gültigen Bereichs
- /impl.cgi:
 - Query startet nicht mit ausg
 - Suffix nicht normal/reset
 - Zeitformat nicht exakt HH:MM:SS
 - Dauer 00:00:00

6. Integrationshinweise (Best Practices)

6.1 Toggle vs. Set

/leds.cgi toggelt den Zustand. Für deterministisches Verhalten in Automationssystemen gilt:

- Vorher Zustand über `/status.xml` abfragen.
- Danach nur dann toggeln, wenn eine Zustandsänderung wirklich nötig ist.

Wenn eine Integration zwingend „set on/off“ braucht, wird für neue Integrationen eher die JSON-REST-API oder das WebSocket-Gateway empfohlen.

6.2 Caching vermeiden

Die Antworten senden `Cache-Control: no-store...` Trotzdem sollten Integratoren Requests nicht über Caches/Proxies leiten, die GET-Requests aggressiv zwischenspeichern.

6.3 Robustheit

- Bei `ERR` zusätzlich den HTTP Statuscode auswerten (400 vs 500).
- Timeouts/retries auf Client-Seite moderat halten (LAN typ. 1–3 Sekunden Timeout, 0–2 Retries).

7. Security-Hinweise

Diese Schnittstelle ist **nicht geschützt**. Jeder Client im Netzsegment kann Relais schalten.

Empfohlene Maßnahmen:

- Nutzung nur im **vertrauenswürdigen LAN/OT-Netz**
- Zugriff per Firewall/VLAN auf definierte Quell-IP/Hosts beschränken
- HTTP Legacy API deaktiviert lassen (`enabled=false`) und nur bei Bedarf aktivieren
- Für Internet-/WAN-nahe Szenarien: stattdessen authentifizierte REST-API + HTTPS verwenden

8. Beispiele

8.1 Beispiel (curl)

```
# Toggle Relais 2
curl -i "http://r-control-ip4.local/leds.cgi?led=2"

# Impuls 30 Sekunden ON auf Relais 1
curl -i "http://r-control-ip4.local/impl.cgi?ausg100:00:30normal"

# XML-Status
curl -s "http://r-control-ip4.local/status.xml"
```

8.2 Beispiel (Home Assistant – REST Sensor Idee)

Die XML-Antwort kann mit Templates geparkt werden (siehe auch `docs/API.md`).

9. Kompatibilität & Änderungen

Die HTTP Legacy API ist eine Bestands-/Legacy-Schnittstelle.

Für Neuintegrationen mit langfristigem Wartungsbedarf wird empfohlen, die authentifizierte JSON-REST-API oder das WebSocket-Gateway zu verwenden (siehe `docs/API.md`).